

C++

01.类基础

02.面向对象

03.类成员进阶

04.类之间的关系

05.继承

07.设计模式单例

08.设计模式-工场-观察者

09.设计模式应用

10.STL

11.线程基础

12.飞机大战

果目的是“通过基类指针调用派生类实现”，并且能在基类声明接口：虚函数多态是标准做法，应该优先用。如果不能在基类声明接口，那只能用类成员函数指针来实现。

概念：相同的行为方式，导致不同的行为结果，同一行语句可以展现多种不同的表现形态。

C++多态：在继承下，父类指针可以指向任何继承于该类的子类对象。有父类的指针进行统一的管理，则父类的指针具有了多种形态，即是多态性。

1.在继承下，父类指针（引用）指向子类对象。（不是子类指针）

2.在父类中定义虚函数，子类重写父类的虚函数。

重写：子类定义了和父类一模一样的虚函数（返回类型，函数名，参数列表）

通过函数名，在编译期就绑定了，也就是确定了调用哪个函数，通过名字调用的普通函数。

virtual函数，编译期匹配的是父类的函数，运行期绑定的是子类的函数，这样才能完成多态。

函数重载

虚函数实现的动态多态

显式的指定父类的类名作用域

__vfptr: void **类型的二级指针，虚函数列表指针，属于对象之中，当类中存在虚函数的时候，编译器默认添加的隐藏的属性。

多个对象中都存在这个指针，多个虚表指针指向的是同一个虚函数列表。

在定义对象调用构造函数的时候，在初始化参数列表中，由编译器默认进行初始化。

虚函数指针会由编译器添加在对象内存空间首地址。

是一个动态的函数指针数组vtable，一个存着虚函数指针的动态数组，数组中每个元素存储的是当前类中虚函数的地址，元素类型 void*。为什么不是函数指针类型，因为这些虚函数不一定是相同的，所以用这种范式的void*。

虚函数列表是属于类的，无论定义多少对象，虚函数表仅有一份，在编译器直接构建出来，数组元素的数量就是虚函数的数量，顺序就是定义顺序。

首先得到对象内存空间中，前面指针的大小的空间，取出里面存储的地址（虚表的地址），通过这个地址间接引用，找到虚表，通过下标直接定位到虚表的具体元素，获取里面的虚函数真正的地址，通过这个函数的入口地址调用虚函数

1.调用流程不同

2.效率不同，虚函数调用流程复杂，效率低。

3.使用场景不同，虚函数使用目的是为了达成多态。一般函数无法实现动态多态。

虚函数列表指针：vfptr，到底是指向的哪个类的序表，取决于定义的是哪个子类的对象，和指向他的指针没有关系。

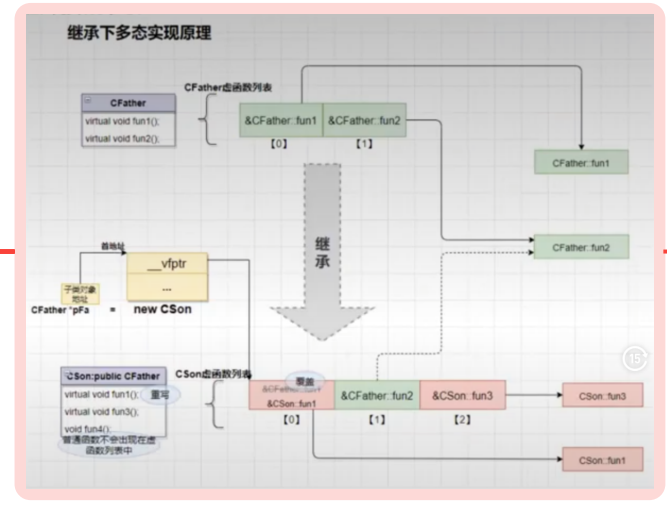
父类：同单个类中一样。

子类：虚表运作流程（在编译期就能完成）

1.继承：子类不但继承父类成员，也会继承父类的虚表。

2.检查重写：如果子类重写父类的虚函数，则在虚表中【原地覆盖】，父类中没有被重写的虚函数，【原地保留】。

3.结尾追加：子类中如果有单独的虚函数，按照顺序在虚表的结尾依次添加。



不要在函数中重新定义默认参数

解决：虚析构，同一般函数一样，也会发生多态。最终调用的是子类的析构函数，自然而然就会调用父类的析构了。

在多态下，父类指针指向子类对象，delete调用析构函数，根据的是指针的类型，所以只是执行了父类的析构。

析构函数会被编译器重新命名，之后可以认为函数名是一样的。所以编译器的角度看，名字是一样的。

子类析构函数和父类的析构函数名不一样，但是属于重写。

纯虚函数无法定义对象

具体类一定要重写抽象类的纯虚函数，否则也无法创建对象。

1.效率：调用虚函数效率低，速度慢。

2.空间：虚函数指针占用空间，多个对象会有多个虚函数指针，虚函数列表会随着继承的层级递增，虚函数表大小只增不减。

3.安全：类中的私有的函数，不能为虚函数，否则会有安全隐患。